

2IMF30: Crash course using the mCRL2 toolset

Jan Martens
j.j.m.martens@tue.nl

Jan Friso Groote
j.f.groote@tue.nl

August 31, 2023

1 Introduction

This document provides a basic introduction in the use of the mCRL2¹ toolset. The toolset is used to model and verify the controller designed in the practical assignment. Gaining some familiarity with the toolset is essential. Therefore, it is required that every participant of the course System Validation (2IMF30) goes through this tutorial individually during the first two weeks of the course.

The document describes a small behavioural model with some properties. The tools can be used to explore and analyse the given model, and to verify the given properties. The core question is whether the properties hold on the model to increase our trust in the quality of both the models and the properties. If the properties do not hold, it may well be that there are serious issues somewhere.

2 Installation

To install the mCRL2 toolset follow the information on the website ². For the purpose in this example the `mcrl2ide.exe` ³ contains all the utilities needed.

However, if models become non trivial and generating state spaces or proving properties becomes hard, the command line tools will be required. In order to run the tools from the command line, depending on the operating system used, you may need to include the `./mCRL2/bin` or `mCRL2.app/Contents/bin` directory to your `PATH`.

Exercise 2.1. Install the toolset for your operating system and open `mcrl-ide.exe`. It should look like Figure 1.

3 The example: A faulty crossing controller

The example in this instruction is a simplified controller of a railroad crossing, inspired by a concrete situation in a small village in The Netherlands. The crossing consists of two components, a signal and a barrier. Each component has a task in the correctly functioning of the crossing.

- The signal indicates to an upcoming train whether it can cross this intersection by showing either a green or a red light.
- The barrier stops upcoming traffic if necessary. It should be closed whenever a train passes.

The full model `crossing_spec.mcrl2`⁴ is also given in the Appendix 5. The model consists of four sections.

¹<https://www.mcrl2.org/>

²<https://www.mcrl2.org/web/home/download.html#>

³<https://www.mcrl2.org/web/user%5fmanual/tools/release/mcrl2ide.html>

⁴<https://gitlab.tue.nl/20200176/system-validation-intro-mcrl2/-/raw/main/crossing%5Fmcrl2project/crossing%5Fspec.mcrl2>

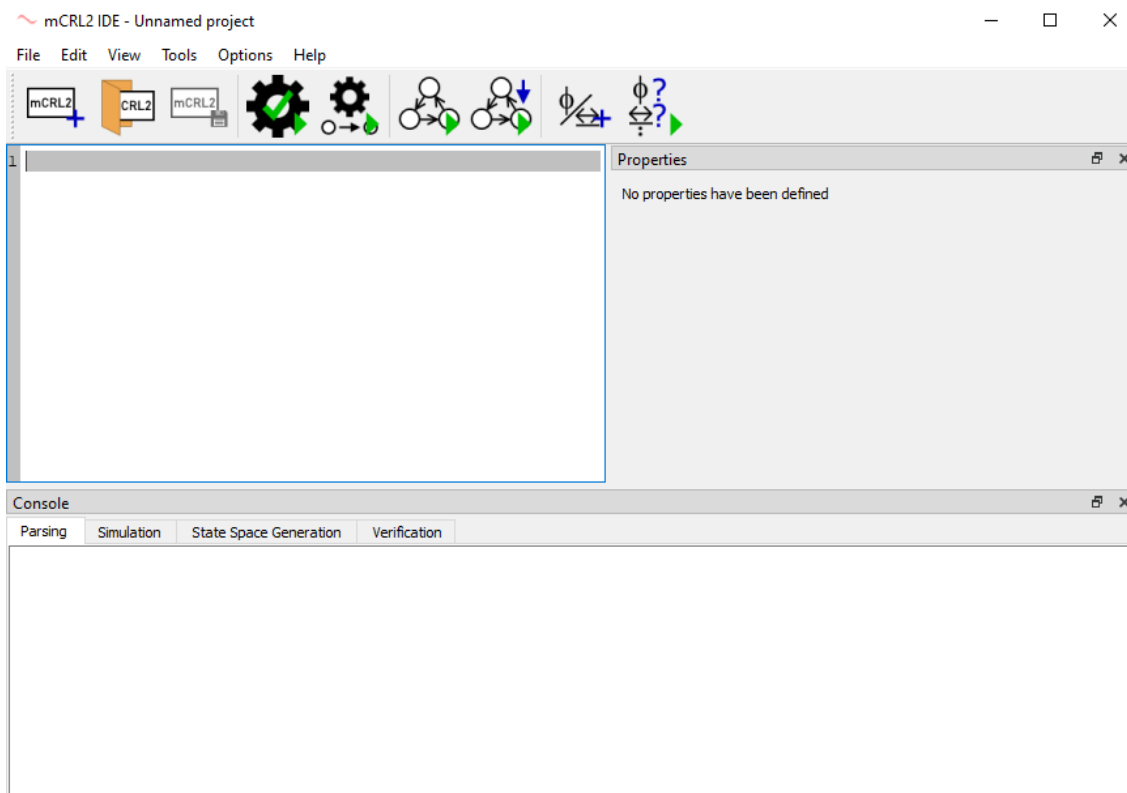


Figure 1: The basic graphical environment to use the mCRL2 toolset

- `sort`, containing the data types used in the model;
- `act`, with the actions that atomically model the crossroad scenario;
- `proc`, defining two processes the crossing controller, and the independent barrier controller; and
- `init`, with the description of the full system, putting the controllers for the crossing and the barrier in parallel.

The `sort` part of this system contain the datatypes `BarrierStatus`, `MovingStatus`, `LightStatus`, which contains the values necessary to distinguish situations of the crossroads.

The `act` part contains all the *actions* in the system. Each action represents an interaction of the system. For example a `greenLight` action models the light turning green, and a `trainArrived` action represents that a train indicates that it wants to cross the intersection. The movement of the barrier is modelled in steps `startOpen`, `startClose` representing the instruction to the barriers to start opening or closing. The actions `barrierOpen`, respectively, `barrierClosed` model that the barrier indicates to the controller that it has successfully opened, respectively closed.

The two processes under `proc` describe the behaviour of the two controllers. They communicate via the actions `sendBarrierStatus`, `receiveBarrierStatus`, `sendBarrierCommand` and `receiveBarrierCommand`.

The initialisation contains two parallel processes. The parallel operator `||` puts the `Crossing` process parallel to the `Barrier` component. The combination of the `comm` and `allow` operator ensure the communication between the components is modelled correctly.

Exercise 3.1. Create a project in the `mcr12ide` (leftmost icon on top). Copy the given model into the main process window. Start simulating the model by pressing the simulation icon at the top of the tool (fifth icon from the left). Execute a few runs, and see how these runs relate to the specification. Generate the state space by the “Show State Space” button and press the “Start” button to get a nicer picture. You can drag states manually, and fix them to the canvas using the right mouse button. The green state is the initial state. Answer the following questions.

- Can a train properly pass the crossing?
- Do you consider the railroad safe?

4 Properties

We consider some simple properties that can be checked against the model. A very simple property is whether the system is free of deadlocks. Although too elementary to express the correctness of a system, it is always a good first step to establish the validity of such simple properties first⁵.

```
% it is always possible to do an action
[true*]<true>true
```

Add a property to the project via the one but rightmost “Add property” button. Added properties will appear in the “Property dock” at the right side of the screen. In this dock you can “Verify” the property against the current specification. Enter the formula and after closing press the green triangle to verify it. The property should be valid. A `W` appears allowing you to find a witness showing that the property is valid. The witness is not very interesting in this case as it consists of the whole state space, as for all states it must be shown that a next step can be done. When a property does not hold, a `C` appears using which a counter example can be obtained.

An important safety condition for the crossway controller is that it should be impossible that the crossing gives a green signal to a train whenever the barrier is open. To verify this property on the model we check a formula in the modal μ -calculus that expresses the property that a `greenLight` is never happening whenever the barrier is open⁶.

⁵Deadlock freedom `no-deadlock.mcf`

⁶Safety `safety.mcf`

```
% After a BarrierOpened action happened and no barrierClosed
% action happened there can't be a greenLight action.
[true*. barrierOpened. !barrierClosed* . greenLight] false
```

Exercise 4.1. Verify this last property on the model by adding it to the project and pressing the play button. Does the property hold? Explain clearly what you observe.

5 Command line

The `mcrl2-ide` does not contain the full strength of the mCRL2 toolset. An overview of all available tools including extensive documentation can be found here⁷.

For instance to generate a labelled transition system (LTS) from the mCRL2 specification which is called `crossing_spec.mcrl2`. We first convert it to a so-called linear process specification (LPS) using `mcrl22lps` and convert this LPS to a LTS using `lps2lts`. We can draw this LTS using `ltsgraph`.

```
# Generate LPS from mcrl2 specification
mcrl22lps crossing_spec.mcrl2 crossing.lps
# Convert LPS to labelled transition system(LTS)
lps2lts crossing.lps crossing.lts
ltsgraph crossing.lts # Tool draws an LTS
```

All tools have various options, such as `--verbose` or `-v` to provide verbose output. Using the flag `--help` or `-h` the main options for each tool are shown. Tools can also be run via a graphical interface, namely `mcrl2-gui`, which provides graphical access to all common options for each tool.

Exercise 5.1. Generate a state space using the command line tools. In order to do so use the commands on the mCRL2 specification.

Exercise 5.2. Explore some other commands for example some of these below

```
lpspp crossing.lps          # Pretty print an LPS
ltsgraph crossing.lts       # Tool draws an LTS
ltsview crossing.lts        # Tool to visualise large LTSs
diagraphica crossing.lts    # Tool to explore data dependencies
```

Appendix A

```
sort BarrierStatus = struct open | close;
    MovingStatus = struct down | up | stable;
    LightStatus = struct green | red;

act sendBarrierStatus, receiveBarrierStatus, commBarrierStatus,
    sendBarrierCommand, receiveBarrierCommand,
    commBarrierCommand: BarrierStatus;
    greenLight, redLight, startOpen, barrierOpen, startClose,
    barrierClosed, barrierOpened, trainArrived, trainPassed;

proc Barrier(status:BarrierStatus, moving:MovingStatus) =
    sendBarrierStatus(status) . Barrier() +
    sum request:BarrierStatus.
```

⁷https://www.mcrl2.org/web/user_manual/tools/tools.html

```

    receiveBarrierCommand(request).
    ((request == close && status == open)
    -> startClose.Barrier(moving = down)
    <> (request == open && status == close)
    -> startOpen.Barrier(moving = up)
    <> Barrier()
    ) +
    (moving == down)
    -> barrierClosed.Barrier(status=close, moving=stable) +
    (moving == up)
    -> barrierOpened.Barrier(status=open, moving=stable);

Crossing(light:LightStatus, trainWaiting:Bool) =
    (!trainWaiting)
    -> trainArrived.Crossing(trainWaiting = true) +
    (trainWaiting)
    -> sum status:BarrierStatus.receiveBarrierStatus(status).
    ((status == close)
    -> greenLight.Crossing(light=green)
    <> sendBarrierCommand(close) . Crossing()) +
    (trainWaiting && light==green)
    -> trainPassed.redLight.
    sendBarrierCommand(open).
    Crossing(light=red, trainWaiting=false);

init allow( { commBarrierCommand, commBarrierStatus, greenLight,
    redLight, startOpen, barrierOpened, startClose,
    barrierClosed, trainPassed, trainArrived },
    comm( { sendBarrierStatus|receiveBarrierStatus -> commBarrierStatus,
    sendBarrierCommand|receiveBarrierCommand ->
    commBarrierCommand },
    Crossing(red, false) || Barrier(open, stable)));

```